```
    if (data.stat == 'ok') {
      var photosDiv = document.getElementById('photos');
      photosDiv.innerHTML = '';
      var photos = data.photos.photo;
      for (var i = 0; i < photos.length; i++) {
        var img = document.createElement('img');
        img.src = photos[i].url_q;
        photosDiv.appendChild(img);
      }
    } else {
      alert(data.message);
    }
  };
```

This code parses the response text into a JavaScript Serialized Object Notation (JSON) object, grabs the images from the object, and displays them on the page. In addition to the response text, the `XMLHttpRequest` object also has properties for the HTTP status, HTTP status text, and methods that retrieve response headers.

## 1.3 Benefits of CORS

The example from the previous section gave you a sense of power of CORS. Now let's turn our attention to some of the benefits CORS provides.

### 1.3.1 Wider audience

If you're building a public API, you want to open up access to as wide an audience as possible. Developers in other languages can easily use native libraries to make API requests. For example, a Python developer can use the httplib2 library to make HTTP requests to any server, regardless of where the request originates. The following snippet shows what a Python request to Flickr looks like. The httplib2 library doesn't care whether the server is CORS-enabled; it indiscriminately makes the request and processes the response, as this sample code shows:

```
import httplib2
h = httplib2.Http(".cache")
resp, content = h.request("http://www.flickr.com/photos/nypl/", "GET")
```

JavaScript developers don't have that advantage because the browser's same-origin policy limits HTTP requests to a single domain. CORS enables JavaScript developers to use an API the same way a developer in another language could.

### 1.3.2 Servers stay in charge

Safety is an important factor when making cross-domain requests. Any cross-domain mechanism needs to be careful not to break the browser's same-origin policy and inadvertently send requests to an unsuspecting server.

CORS achieves this safety by allowing servers to opt-in to CORS. For the request to succeed, the server must use response headers to explicitly acknowledge that the request